

FRI commitment scheme — design and code map

This package implements a polynomial commitment scheme based on **FRI** (Fast Reed–Solomon Interactive Oracle Proof of Proximity), with **DEEP-ALI** combining for batched openings, **k-way coset folding** with coset-per-leaf Merkle layout, and optional **proof-of-work grinding** for soundness amplification.

The implementation is configurable but the defaults follow the recipe used by ethSTARK / Plonky-3: `BlowupFactor = 2`, `FoldingFactor = 8`, `FinalPolynomialMaxLen = 16`, `NumQueries = 20`. The protocol parameters live on `fri.Config` and can be overridden per-application.

This document explains the math, then maps each step onto specific code with file:line references.

1. Notation

Symbol	Meaning
F	the prime field (<code>gnark-crypto/koalabear</code>)
n	base-domain size of an input polynomial (must be a power of two)
q	Reed–Solomon rate = $1 / \text{BlowupFactor}$ (default $1/2$)
N	codeword-domain size, $N = n / p = \text{BlowupFactor} \cdot n$
L	the codeword domain $\langle \omega \rangle \subset F^*$ (order N)
ω	a primitive N -th root of unity, generator of L
k	the folding factor (<code>FoldingFactor</code> , default 8)
ζ	a primitive k -th root of unity, $\zeta = \omega^{(N/k)}$
α_ℓ	the verifier's folding challenge at FRI layer ℓ
β	the DEEP-ALI combiner challenge
z_r, y_r	the r -th open request's evaluation point and claimed value
$Q_r(X)$	the r -th DEEP quotient polynomial
$q(X)$	the β -weighted sum of all Q_r — the input to FRI
g_ℓ	the layer- ℓ codeword ($g_0 = q$ on L)
m	number of query positions per round, <code>Config.NumQueries</code>

All polynomials are stored in **Lagrange-normal form** (evaluations on a domain of the form $\langle \omega \rangle$, in natural index order).

2. The recipe at a glance

We are given some number of polynomials `f_1, ..., f_K` (potentially split across multiple `Commit` calls), each in Lagrange-normal form on its own base domain of size `n_i`. We want to convince a verifier of two things:

1. **Proximity.** Each f_i is close to a codeword of $RS[F, L, n_i]$ — informally, "each f_i really is a low-degree polynomial".
2. **Evaluation.** A list of claims of the form " $f_i(z) = y$ " all hold.

The protocol does this in five phases:

- **Phase A — RS commit.** Each oracle is RS-encoded on the **common** codeword domain L of size N (the same N for every oracle in a single `Prove` call). A coset-per-leaf Merkle tree commits to the encoded values.
- **Phase B — DEEP combiner.** For each open request (f_i, z, y) the prover forms the DEEP quotient $Q(x) = (f_i(x) - y) / (x - z)$. After a Fiat-Shamir challenge β , all quotients are summed into a single codeword $q = \sum \beta^r \cdot Q_r$ on L .
- **Phase C — k-way FRI folding.** q is repeatedly folded by a factor of k using transcript-derived challenges α_ℓ , until the codeword length drops to $\leq \text{FinalPolynomialMaxLen}$. The remaining short codeword is sent in full.
- **Phase D — Optional grinding + queries.** After binding the final polynomial, the prover finds a 64-bit nonce that hashes (with the transcript state) to a value with `Config.GrindingBits` leading zero bits. Then m query indices are drawn from the transcript.
- **Phase E — Verification.** The verifier replays Fiat-Shamir, re-derives every challenge, and per query checks (i) every Merkle path; (ii) that the reconstructed q -coset matches the layer-0 coset; and (iii) that each fold step is consistent across layers all the way to `FinalPolynomial`.

The Reed–Solomon rate q together with m and `GrindingBits` determine soundness; see §10.

3. File map

File	Role
<code>fri.go</code>	<code>Config</code> , <code>Commitment</code> , <code>OpeningProof</code> , <code>Committer</code> (commit phase), <code>Verifier</code> , <code>Bind</code> , <code>Prove</code> , coset-Merkle helpers
<code>fold.go</code>	k-way FRI fold (<code>foldCoset</code> , <code>foldLayer</code> , <code>elementPow</code>)
<code>deep.go</code>	DEEP combiner (<code>computeClaimedValue</code> , <code>buildDEEPCombiner</code>)
<code>query.go</code>	Building per-query Merkle openings + coset data
<code>verify.go</code>	<code>Verifier.VerifyOpening</code> — full proof check
<code>grind.go</code>	Proof-of-work helpers (<code>grindAndBind</code> , <code>verifyAndBindGrinding</code>)
<code>fri_test.go</code>	Unit tests; see also <code>export_test.go</code> for test exports

Reused infrastructure (not in this package):

- `internal/merkle` — Merkle tree (`merkle.New`, `BuildIthLeaf`, `OpenProof`, `Verify`).
- `internal/reedsolomon` — `Encoder.Encode` (Lagrange-on-base → Lagrange-on-codeword via FFT round trip).
- `internal/poly` — `poly.Evaluate` (Lagrange interpolation at an arbitrary point).
- `gnark-crypto/ fiat-shamir` — the transcript.

4. Configuration

`Config` collects every protocol parameter:

```

type Config struct {
    BlowupFactor      int    //  $p^{-1}$ ; default 2
    FoldingFactor      int    //  $k$ ; default 8
    FinalPolynomialMaxLen int    // stop folding when  $|g| \leq$  this; default 16
    NumQueries         int    //  $m$ ; default 20
    MaxCodewordDomainSize uint64 // unify codeword domains across Commits
    GrindingBits       int    // PoW bits; default 0 = off
}

```

`applyDefaults` ([fri.go:60-73](#)) fills zero fields with the defaults declared as `DefaultFRI*` constants ([fri.go:21-25](#)).

`MaxCodewordDomainSize` matters when multiple `Commit` calls cover polynomials of different base sizes: the FRI protocol requires every oracle share **one** codeword domain so that a single `q` can combine all DEEP quotients. Setting `MaxCodewordDomainSize = BlowupFactor · max(n_i)` forces the encoder to lift all oracles to that size. The check in `Commit` is

```

if c.config.MaxCodewordDomainSize > codewordDomainSize {
    codewordDomainSize = c.config.MaxCodewordDomainSize
}

```

([fri.go:188-190](#)). The reverse check — that all oracles agree on `N` — is enforced at the start of `Prove`.

`GrindingBits` is the only parameter the verifier needs to know externally (it's not encoded in the proof — see §11).

5. Phase A — RS encoding and coset-per-leaf Merkle layout

5.1 Reed-Solomon encoding

A polynomial `p` given in Lagrange form on a domain of size `n` is encoded on `L` of size `N = BlowupFactor · n` by going through canonical form:

```
Lagrange[n] --IFFT_n--> canonical[n] --(zero-pad)--> canonical[N]
--FFT_N--> Lagrange[N]
```

This is exactly what `reedsolomon.Encoder.Encode` does. `Committer.encode` ([fri.go:495-509](https://fri.go/495-509)) memoises `fft.Domain` objects per base size so polynomials with different `n_i` in the same `Commit` batch don't repeatedly rebuild domains.

5.2 Coset-per-leaf Merkle layout

For folding factor `k`, the next layer's domain is the image of `L` under $x \mapsto x^k$. The `k` preimages of a single next-layer point form an **arithmetic progression** of indices in `L`:

If $i = j + t \cdot (N/k)$ for any $t \in \{0, \dots, k-1\}$ and fixed $j \in [0, N/k)$, then $(\omega^i)^k = \omega^{\{kj + tN\}} = \omega^{\{kj\}}$ (because $\omega^N = 1$). So all `k` preimages share the same `k`-th power.

A naive Merkle tree that puts one codeword position per leaf would force `k` separate Merkle proofs per query (one per preimage). Putting all `k` preimages in **one leaf** collapses these into a single proof, dropping query communication and verifier work by a factor of `k`.

`buildCosetMerkleTree` constructs this layout:

```
leaf_j = LeafHash( poly_0[j], poly_0[j+N/k], ..., poly_0[j+(k-1)·N/k],
                  poly_1[j], ...,
                  poly_{K-1}[j+(k-1)·N/k] )
```

— `K` polynomials per oracle, `k` positions per polynomial, fixed deterministic order.

For internal FRI **layers** (commit-phase intermediate codewords), there's only one polynomial per layer, so `buildLayerMerkleTree` uses `k` field elements per leaf instead of `K·k`.

The verifier reconstructs leaf bytes via `cosetLeafBytes` (prover and verifier share this helper, which is a one-line wrapper over `marshalElements`).

5.3 Auto-DEEP open per oracle

After the Merkle commitment is bound to the transcript, `Commit` draws one DEEP point `z` per polynomial in the batch, by computing a separate challenge `<challengeName>@deep_<polyName>`:

```
deepBytes, _ := c.transcript.ComputeChallenge(deepChallengeName(challengeName, name))
deepPt.SetBytes(deepBytes)
c.Open(name, deepPt)
```

(fri.go:235-247). This populates `c.openRequests` (one per polynomial per `Commit`). Callers may also `Open` additional points for out-of-domain evaluation claims (the API is exposed for future use; the loom prover currently relies only on auto-DEEP opens).

The verifier mirrors this in `Verifier.Bind`, accumulating the same (`oracleI`, `name`, `point`) triples into `v.deepPoints` so it can reconstruct the DEEP combiner during verification.

6. Phase B — DEEP combiner

6.1 Math

Given a polynomial `f` and a claimed evaluation `y = f(z)`, the **DEEP quotient** is

$$Q(X) := (f(X) - y) / (X - z).$$

Because `f(z) - y = 0`, the numerator vanishes at `X = z` and `Q` is a polynomial of degree `deg(f) - 1` (no leftover remainder). Consequently:

- `Q` is a low-degree polynomial **iff** the claim `y = f(z)` is true.

- For any $X = \omega^i \in L$ ($z \notin L$ is a precondition), $Q(\omega^i)$ can be computed pointwise from $f(\omega^i)$, y , z .

To fold many such quotients into one proximity claim, the prover draws a single combiner challenge β and forms

$$q(X) := \sum_r \beta^r \cdot Q_r(X).$$

If every (f_r, z_r, y_r) claim is correct, q is a polynomial of degree at most $\max_r \deg(f_r) - 1$, and proving low-degree of q simultaneously proves low-degree of every f_r and the correctness of every claimed y_r , up to error in β .

6.2 Code

`buildDEEPCombiner` computes

$$q[i] = \sum_r \beta^r \cdot (\text{codeword_r}[i] - y_r) \cdot (\omega^i - z_r)^{-1} \quad \text{for } i = 0..N-1$$

It precomputes the powers of ω and uses `koalabear.BatchInvert` for amortised inversion. The `req.point IsZero` check ([deep.go:65-69](#)) guards against accidentally choosing a z on the codeword domain (a precondition violation).

The claimed values $y_r = f_r(z_r)$ are computed by `computeClaimedValue`, which delegates to `poly.Evaluate` — Lagrange interpolation at an arbitrary point. They are stored on the proof in `OpeningProof.ClaimedValues` (in the same registration order as `openRequests`) so the verifier can rebuild q at queried positions without recomputing the encoder-side interpolation.

6.3 Why one β suffices for many oracles

`Prove` enforces a single shared codeword domain via the size check at [fri.go:285-298](#). Once every oracle lives on the same L , the β -combination is a single proximity statement — there is no soundness penalty from batching,

only one extra Schwartz-Zippel factor in β .

This is **method 1 (degree padding)** from Boneh's whiteboard treatment: extend everyone to a common degree before combining. We chose this over **method 2 (independent FRI per oracle)** because the loom AIR backend expects to commit many small chunks (one per AIR-quotient piece) and method 1 amortises the FRI fold cost across them.

7. Phase C — k-way FRI folding

7.1 The split

Any polynomial f of degree $< d$ can be uniquely decomposed by powers of X modulo k :

$$f(X) = f_0(X^k) + X \cdot f_1(X^k) + \dots + X^{(k-1)} \cdot f_{(k-1)}(X^k), \text{ each } \deg(f_j) < d/k.$$

For a folding challenge α , define the k -way fold

$$\text{fold}_\alpha(f)(Y) := \sum_{j=0}^{(k-1)} \alpha^j \cdot f_j(Y).$$

$\deg(\text{fold}_\alpha(f)) < d/k$. This is the next layer's polynomial.

7.2 Pointwise computation on a coset

Fix $x_0 \in L$. The k preimages of $Y := x_0^k$ under $x \mapsto x^k$ are $\{\zeta^t \cdot x_0 : t = 0, \dots, k-1\}$ where ζ is a primitive k -th root of unity. Then:

$$f(\zeta^t \cdot x_0) = \sum_j (\zeta^t \cdot x_0)^j \cdot f_j(Y) = \sum_j \zeta^{(t \cdot j)} \cdot (x_0^j \cdot f_j(Y)).$$

Reading the right-hand side as a function of t : $\text{cosetValues}[t]$ are the DFT of $(x_0^j \cdot f_j(Y))_j$ over the k -th roots of unity. Inverting the DFT recovers $c_j := x_0^j \cdot f_j(Y)$, and then

$$\text{fold}_{\alpha}(f)(Y) = \sum_j \alpha^j \cdot f_j(Y) = \sum_j (\alpha / x_0)^j \cdot c_j.$$

So one fold step at one coset costs **one inverse DFT of size k** plus **one Horner evaluation at α/x_0** .

7.3 Code

`foldCoset` :

```
copy(coeffs, cosetValues)
kDomain.FFTInverse(coeffs, fft.DIF) // (1) IDFT over the k-th roots
utils.BitReverse(coeffs)           //      -> coeffs[j] = x_0^j · f_
j(Y)

invBase.Inverse(&cosetBase)          // (2) compute α / x_0
evalAt.Mul(&alpha, &invBase)

// (3) Horner: result = Σ_j coeffs[j] · evalAt^j
for i := k-1; i >= 0; i-- {
    result.Mul(&result, &evalAt)
    result.Add(&result, &coeffs[i])
}
```

`foldLayer` calls `foldCoset` for each of the N/k coset bases $x_0 = \omega^j$, $j \in [0, N/k)$, producing the next layer's codeword.

`runFRICommitPhase` is the loop:

```
for len(g) > config.FinalPolynomialMaxLen && len(g) >= k {
    layerTree := buildLayerMerkleTree(g, k, ...)
    transcript.Bind("fri@layer_<i>", root)
    α := transcript.ComputeChallenge("fri@layer_<i>")
    g = foldLayer(g, α, gen, k)
    gen = elementPow(gen, k) // ω_{ℓ+1} = ω_ℓ^k
}
```

Termination: each iteration shrinks `g` by a factor of `k`, so the loop runs at most $\lceil \log_k(N / \text{FinalPolynomialMaxLen}) \rceil$ times.

7.4 The “no folding happens” edge case

If $N \leq \text{FinalPolynomialMaxLen}$ on entry, the loop body never executes — `finalCodeword = g = q`. This is a legitimate configuration (small input, large `FinalPolynomialMaxLen`) and the verifier handles it specially; see §9.4. The standalone test `TestRoundTripNoLayers` exercises this branch directly.

8. Phase D — Final polynomial, grinding, and queries

After `runFRICommitPhase` returns, `Prove` does three things in order, each separated by a transcript bind so each step's input depends on all previous ones:

1. **Bind the final polynomial.** [fri.go:340-348](#) — the small `finalCodeword` is serialised and bound under `"fri@final"`.
2. **Grinding (optional, §9).** [fri.go:350-358](#) — when `GrindingBits > 0`, find a nonce satisfying the PoW constraint and bind it.
3. **Derive query indices.** [fri.go:361-366](#) — `deriveQueryIndices` draws `m` challenges `"fri@query_<i>i</i>"` and masks each into `[0, N/k)`.

Then `buildQueryProofs` walks each query position once and emits:

- For each oracle: a Merkle proof of the leaf at `j` plus the `K·k` coset values (`OracleOpenings`, `OracleCosetData`).
- For each layer ℓ : a Merkle proof of the leaf at $j_{\ell} = j \bmod (N_{\ell} / k)$ plus the `k` coset values (`LayerOpenings`, `LayerCosetData`).

The leaf-index recurrence between layers is

$$j_{\ell+1} = j_{\ell} \bmod (N_{\ell+1} / k)$$

implemented in [query.go:106-110](#) as `jEll = jEll % nLeavesNext`.

9. Phase E — Verification

`Verifier.VerifyOpening` replays every Fiat-Shamir operation the prover did, then runs per-query checks. It is **self-describing** about every protocol parameter except `GrindingBits` (see §11).

9.1 Replay

The verifier first re-derives every challenge in the same order the prover used:

1. β (from `"fri@combine"`) — [verify.go:53-61](#).
2. α_ℓ (one per `LayerCommitment`, by binding the root and computing `"fri@layer_<ℓ>"`) — [verify.go:64-79](#).
3. The final-polynomial bind under `"fri@final"` — [verify.go:81-99](#).
4. Grinding (if `GrindingBits > 0`): replay the same seed/nonce dance as the prover via `verifyAndBindGrinding`; this both verifies the PoW *and* keeps the transcript in sync — [verify.go:101-106](#).
5. Query indices via `deriveQueryIndices`; the verifier compares them byte-for-byte against `proof.QueryIndices` — [verify.go:108-118](#).

If anything diverges from the prover's transcript order, the indices won't match and verification rejects. This makes Fiat-Shamir mismatches a single-line failure rather than a downstream Merkle failure.

9.2 Inferring k

The proof is **self-describing** about k :

```

switch {
case numLayers > 0 && len(LayerCosetData) > 0:
    k = len(LayerCosetData[0][0])
case len(OracleCosetData) > 0:
    K := Commitments[0].NumPolynomials
    k = len(OracleCosetData[0][0]) / K
default:
    return error
}

```

([verify.go:36-58](#)). The verifier never reads `Config.FoldingFactor`.

9.3 Per-query check (layered path)

For each query at leaf index $j \in [0, N/k)$:

a. DEEP quotient reconstruction. From the supplied oracle coset data and claimed values, the verifier rebuilds the values that q should take at this coset:

$$qCheck[t] = \sum_r \beta^r \cdot (f_r(\omega^{(j+tN/k)}) - y_r) / (\omega^{(j+tN/k)} - z_r), t = 0..k-1.$$

[verify.go:130-174](#).

b. Oracle Merkle paths. Each oracle's path is verified once per query (not once per request) — the same coset bytes serve every claimed open from that oracle. [verify.go:176-188](#).

c. q vs layer-0. `qCheck[t]` must equal `LayerCosetData[qi][0][t]`.
[verify.go:209-213](#).

d. Fold consistency across layers. For each layer ℓ :

- Verify the Merkle proof against `LayerCommitments[ℓ]`.
- Compute `cosetBase =  $\omega_\ell^{(j_\ell)}$` , then `foldCoset(LayerCosetData[qi][ℓ],  $\alpha_\ell$ , cosetBase, kDomain)`.

- For internal layers: this must equal `LayerCosetData[qi][ $\ell+1$ ][ $t_{\ell+1}$ ]`, where $t_{\ell+1} = j_{\ell} / (N_{\ell+1} / k)$ (the position of j_{ℓ} within the next layer's coset).
- For the final layer: this must equal `FinalPolynomial[j $_{\ell}$ ]`.

[verify.go:215-249](#).

The leaf-index recurrence $j_{\ell+1} = j_{\ell} \bmod (N_{\ell+1} / k)$ mirrors `build-QueryProofs`. The accompanying generator update $\omega_{\ell+1} = \omega_{\ell}^k$ is done via `elementPow`.

9.4 Per-query check (0-layer path)

When `numLayers == 0`, the codeword was already small enough that no fold ever ran, and `FinalPolynomial` is the entire `q`. Step (c) is folded directly into the `FinalPolynomial` check, and the fold loop is skipped:

```
for t := range k {
    pos := j + t*nLeaves
    if !qCheck[t].Equal(&proof.FinalPolynomial[pos]) {
        return error
    }
}
```

([verify.go:195-207](#)).

This branch is exercised by `TestRoundTripNoLayers` and `TestNoLayersTamperedFinalPolynomial`.

9.5 What can go wrong, and where it surfaces

Tamper class	Failure surface
Flipped oracle codeword value	qCheck mismatch <i>and</i> Merkle path mismatch (5b/c).
Tampered claimed value	qCheck mismatch (5c).
Tampered final polynomial	Fold mismatch at last layer or 0-layer direct check.
Tampered LayerCommitment root	Re-derived α_ℓ differs $\rightarrow$ Merkle/qCheck cascade fails.
Tampered Merkle sibling	merkle.Verify returns false.
Tampered query index	derivedIndices mismatch (verify.go:113-117).
Tampered grinding nonce	verifyAndBindGrinding rejects, or transcript diverges.
z lands on L	Inversion in DEEP rebuild fails (verify.go:162-164).

Every failure mode has a test in `fri_test.go`; see the "Tampered..." suite.

10. Phase F — Grinding (proof of work)

10.1 Math

Each spot-check query has soundness error $\varepsilon := 1 - \delta$, where δ is the relative Hamming distance to the closest codeword. With $q = 1/2$, the proximity-gap heuristic gives $\delta_{\max} \approx 0.29$, so $\varepsilon \approx 0.71$. With m queries the total error is ε^m , and the bit security is

$$\kappa \approx m \cdot \log_2(1/\varepsilon) \approx 0.49 \cdot m.$$

For a target κ , the prover can do PoW work to "buy" b free bits, pushing the required query count down to

$$m = \lceil (\kappa - b) / \log_2(1/\epsilon) \rceil.$$

For $\kappa = 128$, $b = 24$ reduces m from ≈ 261 to ≈ 213 .

10.2 Construction

The grinding hash is

$H(\text{seed} \parallel \text{nonce_be64})$ — SHA256 in this implementation,

where `seed` is a fresh transcript challenge derived **after** the final polynomial has been bound, but **before** queries are drawn:

```
seed = transcript.ComputeChallenge("fri@grinding_seed")
nonce = smallest n ∈ ℕ such that H(seed ∥ n) has ≥ GrindingBits leading
zeros
transcript.Bind("fri@grinding_nonce", nonce)
```

Binding the nonce back into the transcript ensures the query indices depend on it — a malicious prover cannot grind the seed *and* cherry-pick queries in parallel.

10.3 Code

Helper	Role
<code>grindingHash</code>	<code>SHA256(seed nonce_be64)</code>
<code>hasLeadingZero-Bits</code>	Bit-counting at byte boundaries (covered by <code>TestHasLeadingZeroBitsBoundary</code>)
<code>deriveGrinding-Seed</code>	Pulls the seed from the transcript
<code>bindGrindingNonce</code>	Mirrors prover/verifier transcript bind of the chosen nonce
<code>grindAndBind</code>	Prover-side: search nonce, bind it
<code>verifyAndBind-Grinding</code>	Verifier-side: check the supplied nonce satisfies the bit threshold, then bind

The default `GrindingBits = 0` short-circuits both sides — neither prover nor verifier touches the grinding transcript names, and existing proofs are unaffected.

10.4 Verifier requirement

`GrindingBits` is the **only** parameter the verifier needs externally — unlike `k`, `NumQueries`, etc., it is **not encoded in the proof**. Otherwise a malicious prover could grind 0 bits and claim a strong PoW. The verifier must therefore be configured by its caller via the public field `Verifier.GrindingBits` (or, at the loom integration level, `verifier.WithFRIGrindingBits(n)`).

11. Self-describing proof: what the verifier doesn't need

The proof carries enough information that the verifier infers all of these without external configuration:

Quantity	Inferred from
N (codeword size)	<code>Commitments[0].CodewordDomainSize</code>
K (polys per oracle)	<code>Commitments[oI].NumPolynomials</code>
numLayers	<code>len(LayerCommitments)</code>
numQueries	<code>len(QueryIndices)</code>
k (folding factor)	layer coset length, or oracle coset length / K
FinalPolynomial size	<code>len(FinalPolynomial)</code>

Only `GrindingBits` must be agreed externally (§10.4).

12. Wiring into the loom prover/verifier

The loom-level `prover.Prove` constructs the FRI committer with a config derived from the program ([prover/prover.go](#)):

```
friCfg := fri.Config{
    MaxCodewordDomainSize: fri.DefaultFRIBlowupFactor * maxModuleN,
    GrindingBits:           config.FRIGrindingBits,
}
```

The `config.FRIGrindingBits` value comes from the caller-supplied `prover.WithFRIGrindingBits(n)` option. The verifier mirrors this with `verifier.WithFRIGrindingBits(n)`; both sides default to 0 (no grinding). Tests:

- `TestVerifierWithGrinding` — full prove/verify round-trip with 8-bit PoW.
- `TestVerifierGrindingMismatch` — prover grinds, verifier doesn't, must reject.

Within `internal/commitment/fri/fri_test.go` the corresponding tests are `TestGrindingRoundTrip`, `TestGrindingTamperedNonce`, `TestVerifierGrindingMismatch`, and `TestVerifierMissingGrinding`.

13. Soundness parameter cheat sheet

Setting	κ (bits) at $q = 1/2$	Notes
m=20, b=0 (defaults)	≈ 9.8	Suitable for tests / development only.
m=120, b=0	≈ 59	Mid-range, no grinding.
m=261, b=0	≈ 128	Pure FRI 128-bit conjectured soundness.
m=213, b=24	≈ 128	Cheaper proof, $2^{24} \approx 16\text{M}$ -hash PoW.
m=180, b=40	≈ 128	Smaller proof, $2^{40} \approx 10^{12}$ hashes — significant PoW.

These figures use the standard heuristic and assume the proximity gap (ethSTARK §A.6); see Plonky-3 / RiscZero / ethSTARK for more conservative analyses.

14. Limitations and known gaps

- **External Open requests.** The `Committer.Open` API supports registering arbitrary `(name, point)` pairs but the verifier currently learns about open requests only via `Verifier.Bind` (which only registers auto-DEEP points). The loom prover doesn't use external opens, so this gap is invisible to current users — but a future caller that wants to open at `zeta` (the AIR opening point) would need a verifier-side analogue of `Open`. **Tracked as a follow-up.**
- **Pre-existing vet warning** in `prover/prover.go` (`return copies lock value`) is unrelated to FRI and not introduced by this work; flagged separately.

- **Soundness defaults** (`NumQueries: 20`, `GrindingBits: 0`) are not production parameters; they're sized for fast tests. Pick `m` and `b` based on the security target and proof-size budget for your application.